

CHƯƠNG 4: HỌC SÂU

Học sâu nổi lên như một đối thủ nặng ký trong lĩnh vực này sau 10 năm phát triển bùng nổ của công nghệ tính toán. Do đó, học sâu là một dạng đặc biệt của học máy, với các thuật toán dựa trên cấu trúc và chức năng của bộ não con người.

Học Máy (Machine Learning) VS Học Sâu (Deep Learning)

Học sâu được cho là dạng hiệu quả nhất của học máy. Điều này rất quan trọng vì chúng biết cách tốt nhất để giải quyết vấn đề và cách khắc phục mọi thứ. Sau đây là mô tả về học sâu và học máy:

- **Phụ thuộc vào dữ liệu:** Điểm khác biệt đầu tiên dựa trên đầu ra của DL (Học sâu) và ML (Học máy) khi quy mô dữ liệu tăng lên. Các thuật toán học sâu hoạt động rất tốt khi dữ liệu lớn.
- **Phụ thuộc vào máy móc:** Máy móc cấu hình cao cho phép các thuật toán học sâu hoạt động một cách hoàn hảo. Mặt khác, các thuật toán học máy cũng có thể hoạt động trên các máy tính cấu hình thấp.
- **(Trích xuất đặc trưng):** Các thuật toán học sâu có thể trích xuất các đặc trưng ở cấp độ cao và tìm cách học hỏi từ chúng. Mặt khác, phần lớn các đặc trưng bắt nguồn từ học máy được xác định bởi một chuyên gia.
- **Thời gian thực thi:** Thời gian chạy phụ thuộc vào các tham số thuật toán khác nhau. Học sâu có nhiều tham số hơn các thuật toán học máy. Do đó, thời gian chạy của thuật toán DL nhiều hơn thuật toán ML, đặc biệt là thời gian huấn luyện. Tuy nhiên, thời gian xử lý (dự đoán) của thuật toán DL lại ít hơn ML.
- **Cách tiếp cận giải quyết vấn đề:** Học sâu giải quyết vấn đề một cách toàn diện (end-to-end) trong khi học máy được sử dụng để giải quyết vấn đề theo cách truyền thống, tức là chia nhỏ vấn đề thành các phần.

Các thuật ngữ cơ bản trong Học Sâu

- **Neuron:** Giống như neuron hình thành đặc điểm cơ bản của bộ não chúng ta, một neuron cũng hình thành cấu trúc cơ bản của một mạng nơ-ron. Hãy nghĩ về những gì chúng ta làm khi tiếp nhận thông tin mới. Chúng ta xử lý nó và sau đó tạo ra một đầu ra. Tương tự, trong một mạng nơ-ron (như trong Hình 4.1), đầu vào của một neuron được tiếp nhận, xử lý và tạo ra một đầu ra được gửi đến các neuron khác để xử lý tiếp hoặc trở thành đầu ra cuối cùng.

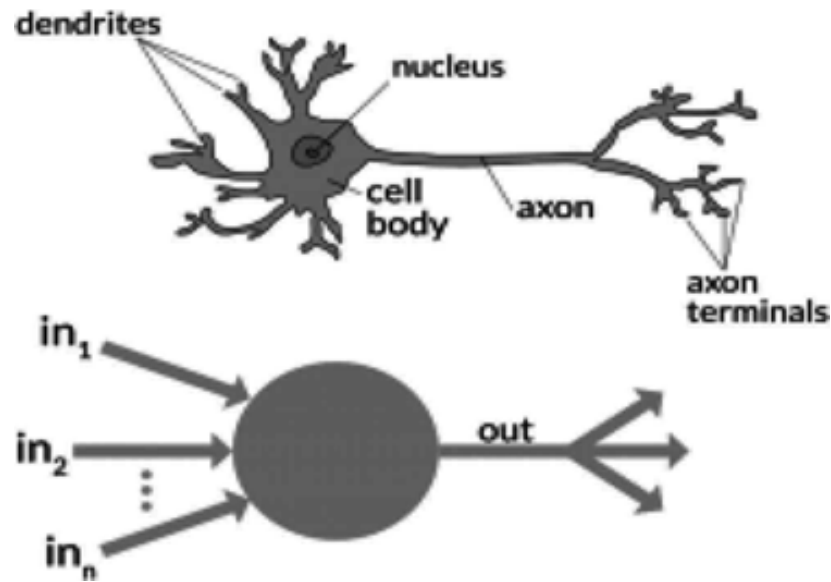


Fig. 4.1 Structure of Neuron

- **Trọng số (Weights):** Đầu vào được nhân với một trọng số khi được áp dụng cho neuron. Ví dụ, nếu một neuron có hai đầu vào, trọng số sẽ được gán cho mỗi đầu vào. Chúng ta khởi tạo trọng số một cách ngẫu nhiên và thay đổi các trọng số này trong quá trình huấn luyện mô hình. Sau khi huấn luyện, mạng nơ-ron sẽ gán trọng số lớn hơn cho đầu vào mà nó thấy quan trọng hơn. Trọng số bằng không có nghĩa là yếu tố cụ thể đó không quan trọng.
- **Độ lệch (Bias):** Ngoài các trọng số, đầu vào còn được áp dụng thêm một thành phần tuyến tính khác, gọi là độ lệch (bias). Kết quả sẽ có dạng $a * W_1 + bias$ sau khi thêm độ lệch này. Đây là thành phần tuyến tính cuối cùng của phép biến đổi dữ liệu.
- **Hàm kích hoạt (Activation Function):** Một hàm phi tuyến tính được giới thiệu sau khi thành phần tuyến tính được tính toán. Hàm kích hoạt chuyển đổi tín hiệu đầu vào thành tín hiệu đầu ra. Hàm này được áp dụng cho mỗi quan hệ tuyến tính. Kết quả sau khi kích hoạt sẽ có dạng $f(a * W_1 + b)$, trong đó f là hàm kích hoạt. Trong sơ đồ (Hình 4.2), các đầu vào là X_1 đến X_n và các trọng số tương ứng là Wk_1 đến Wk_n . Chúng ta có một độ lệch (bias) là bk . Đầu tiên, các trọng số được nhân với đầu vào tương ứng rồi cộng lại với nhau. Chúng ta gọi đây là u .

$$u = \sum(w * x) + b$$

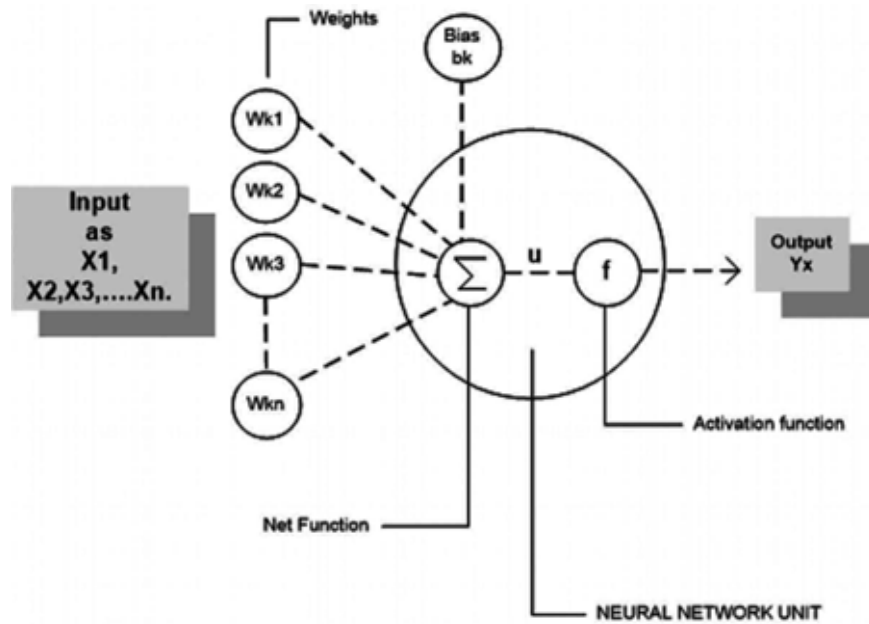


Fig. 4.2 Activation Function

Hàm kích hoạt được áp dụng cho u , tức là $f(u)$, và chúng ta nhận được đầu ra cuối cùng của nơ-ron là:

$$y_k = f(u)$$

- **Các hàm kích hoạt** được sử dụng phổ biến nhất là sigmoid, ReLU và SoftMax.
- **Lớp đầu vào/Lớp đầu ra/Lớp ẩn (Input/Output/Hidden layer):** Đúng như tên gọi, lớp đầu vào là lớp đầu tiên của mạng nhận đầu vào. Lớp đầu ra là lớp xuất kết quả hoặc là lớp cuối cùng của mạng. Các lớp truyền tải của mạng là các lớp ẩn. Các lớp ẩn này thực hiện các nhiệm vụ khác nhau trên dữ liệu đầu vào và chuyển đầu ra đến lớp tiếp theo. Các lớp đầu vào và đầu ra có thể được nhìn thấy trong khi các lớp trung gian bị “che khuất” (Hình 4.3).

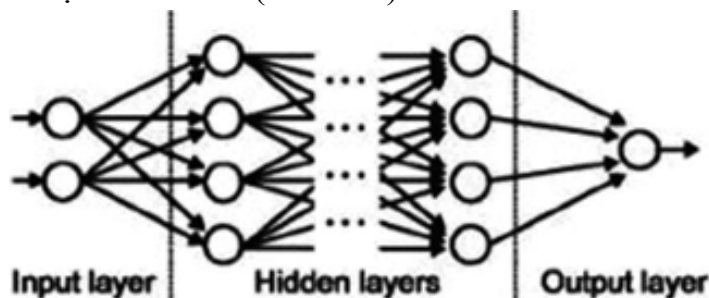


Fig. 4.3 Representation of layers

- **Perceptron đa lớp (Multilayer Perceptron):** Một nơ-ron đơn lẻ không thể hoàn thành các nhiệm vụ phức tạp. Do đó, chúng ta tạo ra các đầu ra cần thiết bằng các “ngăn xếp” nơ-ron. Trong mạng đơn giản nhất, chúng ta có một lớp đầu vào, một lớp ẩn và một lớp đầu ra. Mỗi lớp có nhiều nơ-ron, và tất cả các nơ-ron trong mỗi lớp được kết nối với các nơ-ron của lớp tiếp theo. Các mạng này cũng có thể được gọi là kết nối đầy đủ (fully connected).

- **Hàm chi phí (Cost Function):** Khi chúng ta tạo một mạng, mạng cố gắng ước tính xem hiệu suất của nó gần với giá trị thực tế đến mức nào. Chúng ta sử dụng hàm chi phí/mất mát (cost/loss function) để tính toán độ chính xác này. Nếu mạng mắc lỗi, hàm chi phí sẽ “phạt” nó. Mục tiêu của chúng ta là nâng cao khả năng dự đoán và giảm lỗi, do đó giảm chi phí. Hàm chi phí hoặc mất mát tối thiểu tương ứng với hiệu suất hiệu quả nhất. Nếu hàm chi phí được định nghĩa là lỗi toàn phương trung bình (mean squared error), nó có thể được viết là:

$$C = (1/m) * \sum (y - a)^2$$

trong đó m là số lượng đầu vào huấn luyện, a là giá trị dự đoán và y là giá trị thực tế. Chu trình học tập chính là việc giảm chi phí.

- **Giảm theo Gradient (Gradient Descent):** Đây là một thuật toán tối ưu hóa để giảm chi phí. Để hình dung, hãy tưởng tượng bạn đi xuống một ngọn đồi, bạn sẽ đi xuống dần từng bước thay vì chạy thẳng xuống. Để tìm cực tiểu cục bộ của một hàm, chúng ta thực hiện các bước tỷ lệ thuận với giá trị âm của gradient của hàm, coi điểm dưới cùng là điểm chi phí tối thiểu.
- **Tốc độ học (Learning Rate):** Trong mỗi lần lặp, tốc độ học được định nghĩa là mức độ giảm thiểu chi phí. Tốc độ mà chúng ta đi xuống điểm tối thiểu của hàm chi phí đơn giản là tốc độ học. Chúng ta nên cẩn thận lựa chọn tốc độ học, vì nếu quá cao, giải pháp tối ưu có thể bị bỏ qua, hoặc nếu quá thấp, mạng có thể mất rất nhiều thời gian để hội tụ (Hình 4.5).

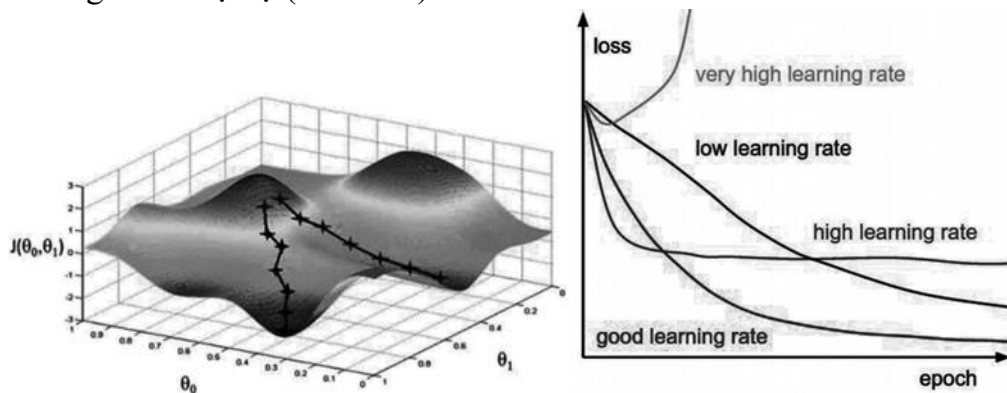


Fig. 4.5 Representation of Learning Rate

- **Lan truyền ngược (Backpropagation):** Khi xây dựng một Mạng nơ-ron, chúng ta gán trọng số và độ lệch ngẫu nhiên cho các nút. Chúng ta có thể đo lỗi của mạng sau một lần lặp. Lỗi này sau đó được lan truyền ngược lại mạng cùng với gradient của hàm chi phí để điều chỉnh các trọng số. Các trọng số này được sửa đổi để giảm lỗi trong các lần lặp tiếp theo. Sự thay đổi trọng số này được gọi là lan truyền ngược.
- **Lô (Batches):** Trong quá trình huấn luyện, chúng ta chia đầu vào thành nhiều phần (lô) có kích thước bằng nhau một cách ngẫu nhiên, thay vì gửi toàn bộ đầu vào trong một lần. Việc huấn luyện dữ liệu theo lô giúp mô hình dễ mở rộng hơn.
- **Kỷ nguyên (Epochs):** Một epoch (kỷ nguyên) được định nghĩa là một lượt lan truyền thuận và ngược của *toàn bộ* dữ liệu đầu vào (qua tất cả các lô). Bạn có thể

chọn số lượng epoch để huấn luyện mạng của mình. Nhiều epoch hơn có khả năng cho độ chính xác cao hơn, nhưng sẽ mất nhiều thời gian hơn để hội tụ.

- **Dropout:** Dropout là một kỹ thuật điều chuẩn (regularization) giúp tránh hiện tượng mạng bị quá khớp (overfitting). Đúng như tên gọi, một số nơ-ron trong lớp ẩn bị “loại bỏ” một cách ngẫu nhiên trong quá trình huấn luyện. Điều này có nghĩa là việc huấn luyện diễn ra trên nhiều kiến trúc khác nhau với các tổ hợp nơ-ron khác nhau. Bạn có thể coi dropout như một quy trình “tổ hợp” (ensemble), tạo ra kết quả cuối cùng bằng cách sử dụng đầu ra của nhiều mạng (Hình 4.6).

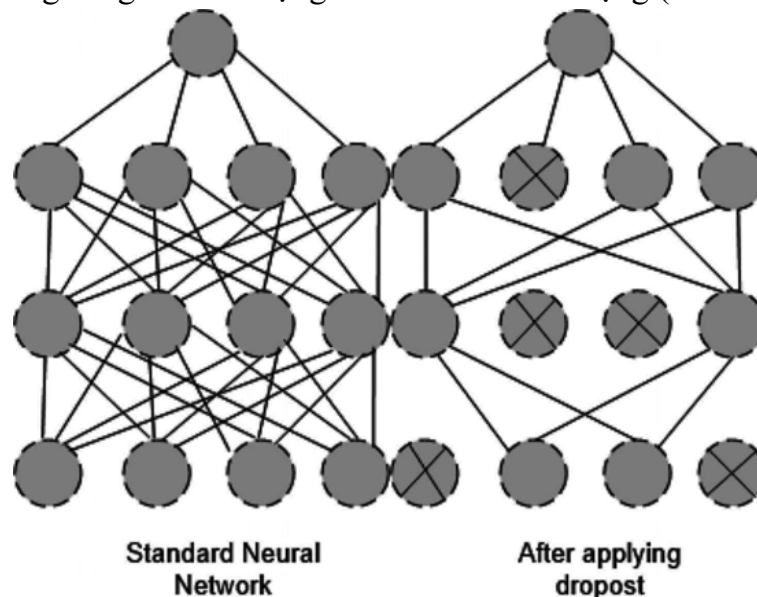


Fig. 4.6 Demonstrating working of dropout

- **Chuẩn hóa theo lô (Batch Normalization):** Chuẩn hóa theo lô có thể được xem như một quy trình giống như chúng ta đặt một điểm kiểm soát đặc biệt trên một dòng sông. Điều này được thực hiện để đảm bảo phân phối dữ liệu (distribution) giống như những gì lớp tiếp theo mong đợi. Khi huấn luyện mạng nơ-ron, các trọng số thay đổi sau mỗi pha giảm gradient. Điều này làm thay đổi dạng thức dữ liệu được gửi đến lớp tiếp theo. Tuy nhiên, lớp tiếp theo lại mong đợi phân phối dữ liệu giống như những gì nó đã thấy trước đó. Vì vậy, trước khi gửi dữ liệu sang giai đoạn tiếp theo, chúng ta chuẩn hóa (normalize) dữ liệu một cách đặc biệt.
- **Huấn luyện (Training):** Các trọng số bắt đầu bằng các giá trị ngẫu nhiên. Khi mạng nơ-ron dần làm quen với loại dữ liệu đầu vào, mạng sẽ thay đổi trọng số dựa trên bất kỳ lỗi phân loại nào phát sinh từ các trọng số trước đó. Khi mạng đã học xong, chúng ta có thể dự đoán kết quả cho đầu vào liên quan.

Mạng Nơ-ron (Neural Networks)

Mạng nơ-ron là các thiết bị tính toán song song nhằm cố gắng xây dựng một mô hình máy tính mô phỏng bộ não. Mục tiêu chính là phát triển một hệ thống cho phép thực hiện

một số tác vụ máy tính nhanh hơn các hệ thống truyền thống. Các nhiệm vụ này bao gồm nhận dạng và xác định mẫu, ước tính, tối ưu hóa và phân cụm dữ liệu.

Mạng Nơ-ron Nhân tạo (Artificial Neural Networks - ANN)

Mạng nơ-ron nhân tạo (ANN) là một hệ thống tính toán hiệu quả với chủ đề cốt lõi được lấy từ sự tương đồng với các mạng nơ-ron sinh học. ANN còn được gọi là Hệ thống Nơ-ron Nhân tạo, Hệ thống Xử lý Phân tán Song song và Hệ thống Kết nối (Connectionist Systems).

ANN có được một tập hợp lớn các đơn vị (unit) được liên kết theo một mẫu nào đó để cho phép tiếp xúc giữa chúng. Các đơn vị như vậy là các bộ xử lý đơn giản hoạt động song song, còn được gọi là các nút (node) hoặc nơ-ron (neuron). Mỗi nơ-ron được liên kết với một nơ-ron khác bằng một liên kết kết nối. Mỗi liên kết kết nối được liên kết với một trọng số mang thông tin tín hiệu đầu vào. Đây là kiến trúc hữu ích nhất để nơ-ron giải quyết một vấn đề cụ thể, vì trọng số thường kích hoạt hoặc ức chế tín hiệu. Trạng thái bên trong của mỗi nơ-ron được gọi là tín hiệu kích hoạt. Sau khi kết hợp các tín hiệu đầu vào và quy tắc kích hoạt, các tín hiệu đầu ra được tạo ra có thể được truyền đến các đơn vị khác.

Mô hình Mạng Nơ-ron Nhân tạo: Sơ đồ (Hình 4.7) biểu diễn mô hình ANN tổng quát và quy trình của nó. Đối với mô hình tổng quát trên, đầu vào ròng (net input) có thể được tính như sau:

$$y_{in} = x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 \dots x_m \cdot w_m$$

Tức là, Đầu vào ròng $y_{in} = \sum m_i x_i \cdot w_i$

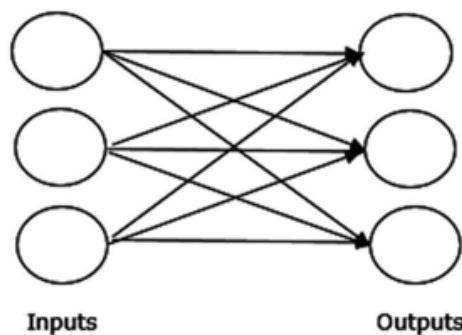


Fig. 4.7 Single Layer Feed forward Network

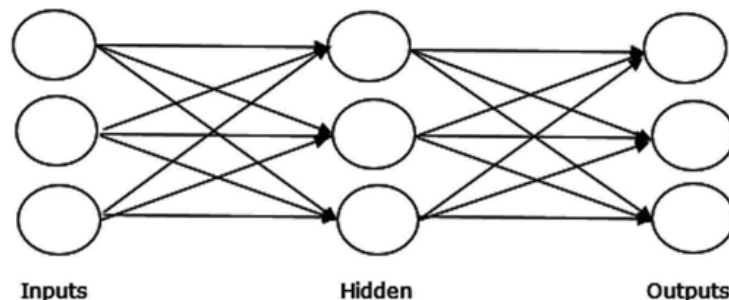


Fig. 4.8 Multiple Layer Feedforward Network

Đầu ra có thể được tính bằng cách áp dụng hàm kích hoạt lên đầu vào rỗng.

$$Y = F(y_{in})$$

Quá trình xử lý của ANN phụ thuộc vào ba khối xây dựng sau đây:

1. Cấu trúc liên kết mạng (Network Topology) Sự sắp xếp của một mạng với các nút và đường liên kết của nó là cấu trúc liên kết mạng. ANN có thể được phân loại theo các dạng sau đây dựa trên cấu trúc liên kết:

- **Mạng truyền thẳng (Feedforward Network):** Đây là một mạng không có vòng lặp, nơi tín hiệu chỉ có thể truyền theo một hướng từ đầu vào đến đầu ra.
 - **Mạng truyền thẳng một lớp:** Chỉ có một lớp trọng số; lớp đầu vào được kết nối đầy đủ với lớp đầu ra.
 - **Mạng truyền thẳng đa lớp:** Có nhiều hơn một lớp trọng số, bao gồm một hoặc nhiều lớp ẩn nằm giữa lớp đầu vào và lớp đầu ra.
- **Mạng phản hồi (Feedback Network):** Mạng này có các đường dẫn phản hồi, cho thấy tín hiệu sẽ chạy qua các vòng lặp theo bất kỳ hướng nào. Điều này làm cho nó trở thành một hệ thống động phi tuyến tính.
 - **Mạng Hồi quy (Recurrent Networks):** Đây là các mạng phản hồi vòng kín.
 - **Mạng Hồi quy Đầy đủ:** Tất cả các nút đều được kết nối với tất cả các nút khác (Hình 4.9).

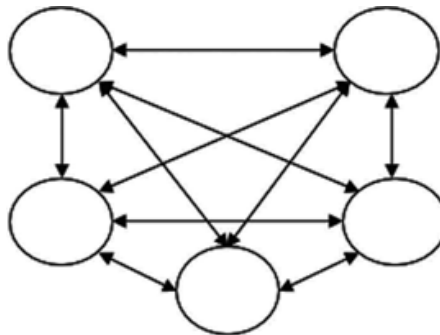


Fig. 4.9 Fully Recurrent Network

- **Mạng Jordan:** Một mạng mạch kín nơi đầu ra được truyền đến đầu vào dưới dạng phản hồi (Hình 4.10).

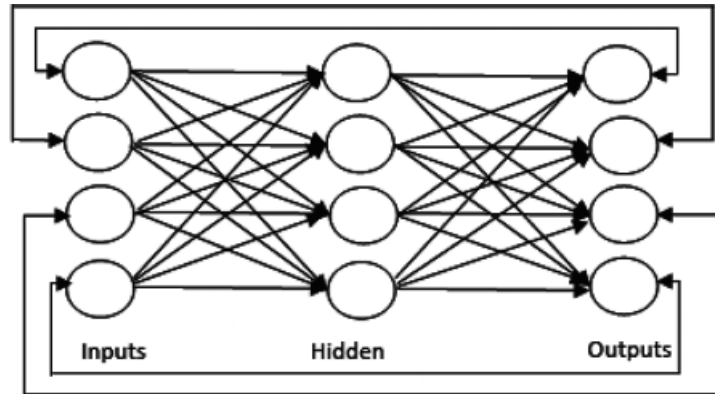


Fig. 4.10 Jordan Network

2. Điều chỉnh Trọng số hoặc Học tập Phương pháp sửa đổi trọng số của các kết nối giữa các nơ-ron được gọi là học tập. Việc học của ANN có thể được phân thành ba nhóm:

- **Học có giám sát (Supervised Learning):** Diễn ra dưới sự hướng dẫn của “giáo viên”. Vector đầu vào được đưa vào mạng, tạo ra một vector đầu ra. Vector này được so sánh với vector đầu ra mong muốn. Nếu chúng khác nhau, một tín hiệu lỗi sẽ được tạo ra. Các trọng số được điều chỉnh dựa theo tín hiệu lỗi này cho đến khi đầu ra thực tế khớp với đầu ra mong muốn.

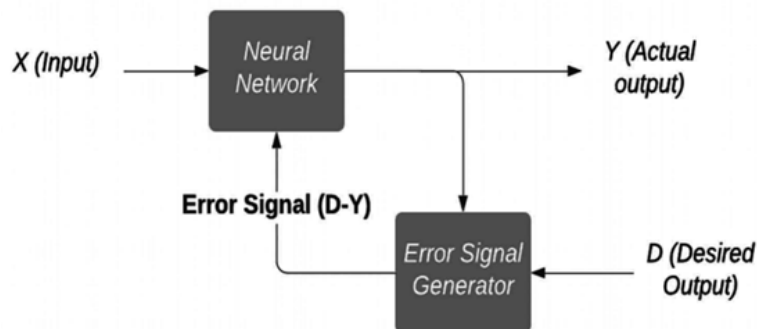


Fig. 4.11 Dependent Learning Process

- **Học không giám sát (Unsupervised Learning):** Được thực hiện mà không có sự giám sát. Các vector đầu vào có cùng dạng được gộp vào các cụm (cluster). Mạng phải tự khám phá các mẫu và đặc trưng của đầu vào.

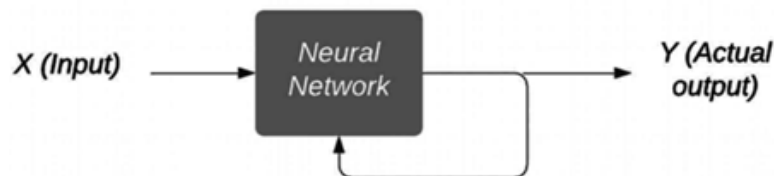


Fig. 4.12 Individual Learning Process

- **Học tăng cường (Reinforcement Learning):** Được sử dụng để củng cố hoặc cải thiện mạng thông qua dữ liệu “phê bình” (hoặc phản hồi). Quá trình này tương tự như học có giám sát, nhưng chúng ta có thể có ít thông tin hơn nhiều. Mạng nhận được một số đầu vào từ “môi trường”. Phản hồi này không mang tính mô tả chi

tiết, vì không có “người hướng dẫn”. Mạng sẽ thay đổi các trọng số để tối ưu hóa thông tin trong tương lai sau khi nhận được phản hồi.

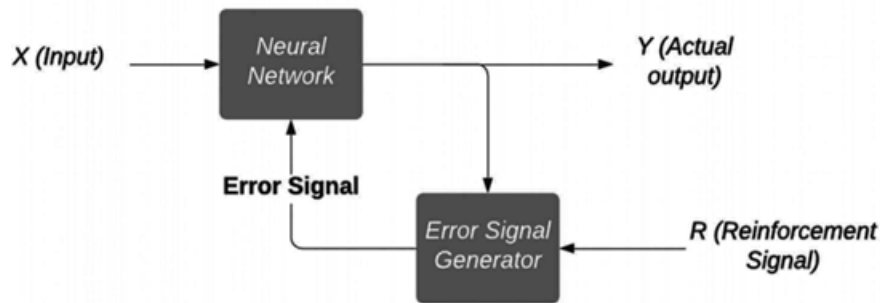


Fig. 4.13 Reinforcement Learning

3. Các hàm kích hoạt (Activation Functions)

Hàm kích hoạt có thể được định nghĩa là lực hoặc nỗ lực bổ sung tác động lên đầu vào để có được đầu ra chính xác. Dưới đây là một số hàm kích hoạt quan trọng:

- **Hàm kích hoạt tuyến tính:** Còn được gọi là hàm đồng nhất (identity function) vì nó không chỉnh sửa dữ liệu. Công thức: $F(x) = x$
- **Hàm kích hoạt Sigmoid:** Có hai loại:
 - **Hàm Sigmoid nhị phân:** Chỉnh sửa đầu vào trong khoảng từ 0 đến 1. Công thức: $F(x) = \text{sigm}(x) = 1 \div (1 + \exp(-x))$
 - **Hàm Sigmoid song cực:** Điều chỉnh đầu vào trong khoảng từ -1 đến 1. Công thức: $F(x) = \text{sigm}(x) = 2 \div (1 + \exp(-x)) - 1$

Thách thức trong ANN Bước đầu tiên trong việc giải quyết câu hỏi phân loại hình ảnh bằng ANN là biến đổi hình ảnh 2D thành vector 1D trước khi huấn luyện. Có hai bất tiện:

- Khi kích thước ảnh tăng, số lượng tham số có thể huấn luyện tăng lên đáng kể.
- ANN làm mất các đặc điểm không gian (spatial characteristics) của hình ảnh.
- ANN không thể ghi lại thông tin tuần tự (sequential information) để xử lý dữ liệu chuỗi.

Mạng Nơ-ron Tích chập (Convolutional Neural Network - CNN)

Mạng nơ-ron tích chập (CNN) cũng giống như mạng nơ-ron thông thường vì nó bao gồm các nơ-ron có trọng số học được và các đặc điểm độ lệch (bias). Các mạng nơ-ron thông thường bỏ qua cấu trúc dữ liệu đầu vào và chuyển đổi tất cả dữ liệu thành một mảng 1-D.

Vấn đề này được CNN giải quyết dễ dàng. Nó tính đến cấu trúc 2D của hình ảnh khi xử lý chúng, cho phép chúng trích xuất các thuộc tính dành riêng cho hình ảnh. Mục tiêu chính của CNN là chuyển từ dữ liệu hình ảnh thô ở lớp đầu vào sang lớp (class) chính xác ở lớp đầu ra.

Tổng quan kiến trúc của CNN Kiến trúc CNN được cấu trúc với các nơ-ron theo ba chiều: chiều rộng, chiều cao và chiều sâu. Mỗi nơ-ron của lớp hiện tại được liên kết với một mảng nhỏ (receptive field) của đầu ra lớp trước đó.

Các lớp được sử dụng để xây dựng CNN

- **Lớp đầu vào (Input Layer):** Nhận dữ liệu hình ảnh thô.
- **Lớp tích chập (Convolutional Layer):** Đây là thành phần trung tâm của CNN, nơi thực hiện hầu hết các phép tính. Lớp này đo lường các phép tích chập (convolutions) trên đầu vào.
- **Phần đệm (Padding):** Khi bộ lọc không khớp hoàn toàn với hình ảnh đầu vào, chúng ta có hai lựa chọn:
 - Đệm ảnh bằng các số 0 (zero padding) để vừa vặn.
 - Xóa thành phần ảnh không khớp. Đây được gọi là “đệm hợp lệ” (valid padding), chỉ giữ lại phần hợp lệ của hình ảnh.
- **Lớp ReLU (Rectified Linear Unit):** Dùng để kích hoạt đầu ra của lớp trước. Nó làm tăng tính phi tuyến tính của mạng.
- **Lớp Gộp (Pooling Layer):** Gộp giúp bảo tồn các phần thiết yếu khi chúng ta tiến sâu vào mạng. Lớp gộp hoạt động trên từng lát (slice) đầu vào một cách độc lập và thay đổi kích thước không gian của nó. Nó thường được gọi là lấy mẫu con (subsampling) hoặc lấy mẫu giảm (down sampling).
 - Các dạng gộp không gian có thể là: Gộp Cực đại (Max Pooling) , Gộp Trung bình (Average Pooling) , và Gộp Tổng (Sum Pooling).
 - Gộp cực đại lấy phần tử lớn nhất từ bản đồ đặc trưng đã được chỉnh lưu (rectified).
- **Lớp Kết nối Đầy đủ/Lớp Đầu ra (Fully Connected Layer):** Chúng ta “làm phẳng” (flatten) ma trận và đưa nó vào một lớp kết nối đầy đủ (FC layer), tương tự như một mạng nơ-ron. Các giá trị đầu ra ở lớp cuối cùng được tính toán bởi lớp này. Cuối cùng, chúng ta có một hàm kích hoạt để phân loại đầu ra, chẳng hạn như SoftMax hoặc sigmoid.

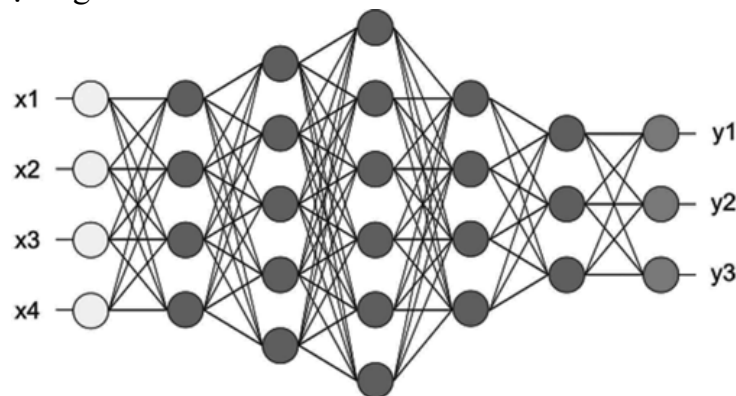


Fig. 4.14 After pooling layer, flattened as FC layer

Mạng Nơ-ron Hồi quy (Recurrent Neural Network - RNN)

Mạng nơ-ron hồi quy (RNN) là một lớp mạng nơ-ron nhân tạo, nơi các kết nối giữa các đơn vị tạo thành một đồ thị có hướng dọc theo một chuỗi (sequence). Điều này cho phép chúng thể hiện các hành vi phức tạp theo thời gian.

Bộ nhớ trong của RNN có thể được sử dụng để xử lý các chuỗi đầu vào. Điều này khiến chúng có thể áp dụng cho các tác vụ như nhận dạng chữ viết tay hoặc ngôn ngữ không phân đoạn, có liên quan.

RNN phát huy tác dụng khi mô hình yêu cầu “bối cảnh” (context) để cung cấp đầu ra dựa trên dữ liệu. Tương tự như khi bạn xem phim, bạn hiểu được diễn biến vì bạn nhớ những gì đã xem trước đó. RNN cũng “ghi nhớ” mọi thứ. Trong RNN, tất cả các đầu vào đều được kết nối. Để làm điều này, RNN tạo ra các mạng có vòng lặp (loop) cho phép thông tin tồn tại dai dẳng (persist) (Hình 4.16).

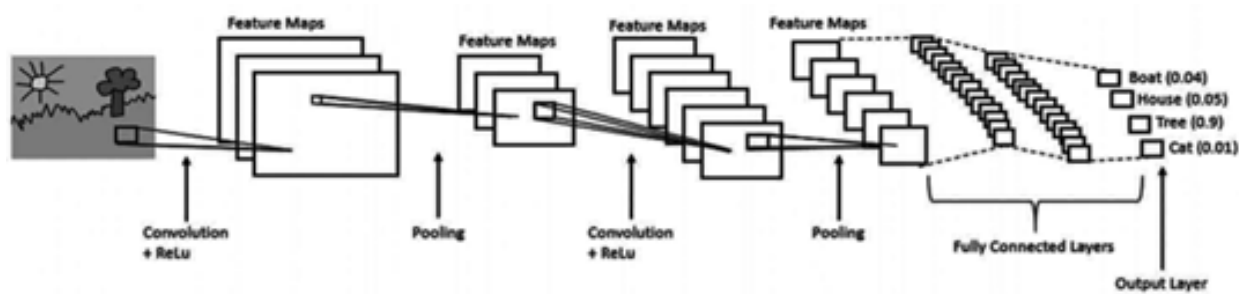


Fig. 4.15 Complete CNN architecture.

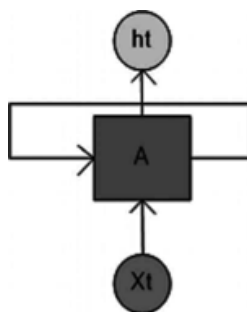


Fig. 4.16 RNN with a LOOP

Nếu “trải dài” (unrolled) cấu trúc (Hình 4.17), bạn có thể thấy: Đầu tiên, $x(0)$ được lấy từ chuỗi đầu vào và tạo ra $h(0)$. Sau đó, $h(0)$ và $x(1)$ là đầu vào cho bước tiếp theo. Tương tự, $h(1)$ cùng với $x(2)$ là đầu vào cho bước sau đó, cứ thế tiếp diễn.

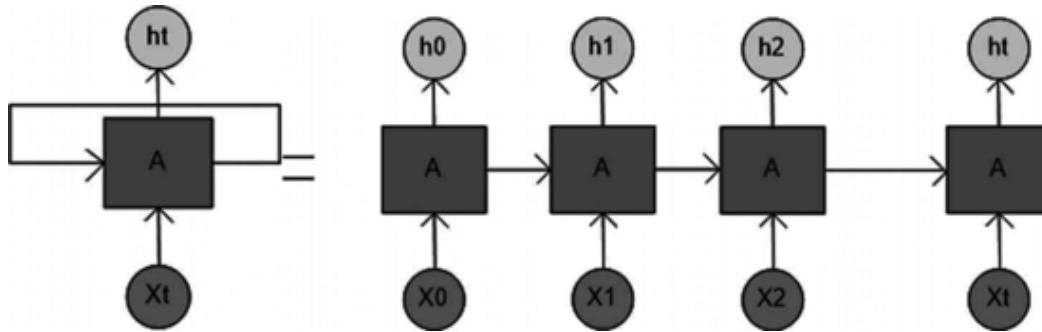


Fig. 4.17 Unrolled Structure of RNN

Các loại RNN RNN cho phép hoạt động trên các chuỗi vector: chuỗi đầu vào, chuỗi đầu ra, hoặc cả hai. (Xem Hình 4.18)

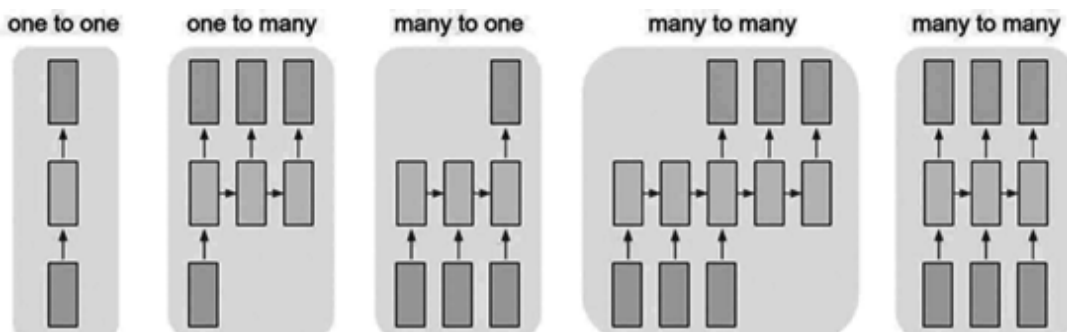


Fig. 4.18 Types Of RNN

- **Một-một (One to One):** Mạng nơ-ron thông thường. Xử lý đầu vào kích thước cố định sang đầu ra kích thước cố định. Ví dụ: Phân loại hình ảnh.
- **Một-nhiều (One to many):** Nhận đầu vào kích thước cố định và cung cấp chuỗi dữ liệu làm đầu ra. Ví dụ: Gắn thẻ ảnh (Image captioning).
- **Nhiều-một (Many to One):** Nhận chuỗi thông tin làm đầu vào và đưa ra một đầu ra cố định. Ví dụ: Phân tích cảm xúc (sentimental analysis).
- **Nhiều-nhiều (Many to Many):** Nhận một chuỗi thông tin làm đầu vào và xử lý để tạo ra một chuỗi đầu ra. Ví dụ: Dịch máy.
- **Nhiều-nhiều (đồng bộ):** Chuỗi đầu vào và đầu ra được đồng bộ hóa. Ví dụ: Phân loại video (video grading) nơi mỗi khung hình video được gán nhãn.

Thách thức của RNN Các RNN sâu (RNN có nhiều bước thời gian) thường bị ảnh hưởng bởi các vấn đề “gradient biến mất” (vanishing gradient) và “gradient bùng nổ” (exploding gradient), vốn là đặc điểm chung của tất cả các loại mạng nơ-ron.

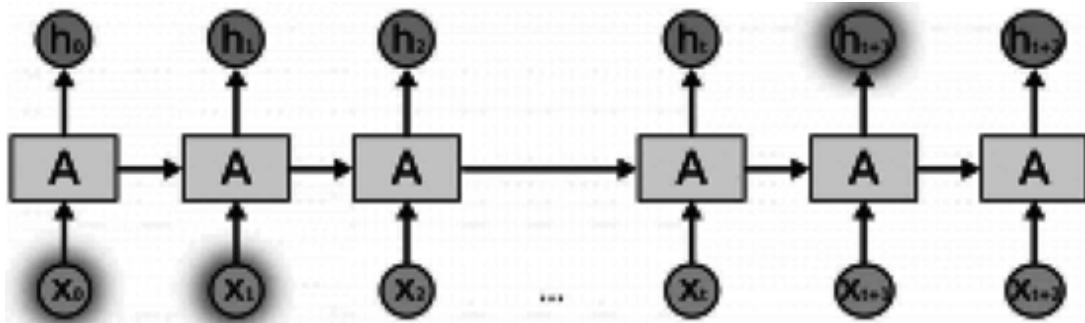


Fig. 4.19 Recurrent Neural Network

Sự khác biệt giữa ANN, CNN & RNN

Bảng sau đây mô tả sự khác biệt giữa Mạng Nơ-ron Nhân tạo, Mạng Nơ-ron Tích chập và Mạng Nơ-ron Hồi quy:

	ANN	CNN	RNN
Dữ liệu	Dữ liệu dạng bảng (Tabular)	Dữ liệu hình ảnh	Dữ liệu chuỗi (Sequence)
Kết nối Hồi quy	Không	Không	Có
Chia sẻ Tham số	Không	Có	Có
Mối quan hệ Không gian	Không	Có	Không
Gradient Biến mất/Bùng nổ	Có	Có	Có

Tóm tắt

Trong chương này, chúng ta đã nghiên cứu về những điều cơ bản của học sâu, bao gồm các thuật ngữ hoặc các khái niệm cơ bản chúng ta đề cập khi sử dụng học sâu hoặc bất kỳ thuật toán nào khác như độ lệch (bias), tốc độ học (learning rate), v.v., và chúng ta cũng đã nghiên cứu về mạng nơ-ron nhân tạo, Mạng nơ-ron Tích chập và Mạng nơ-ron Hồi quy với cấu trúc cơ bản của chúng, có thể được sử dụng trong các tình huống khác nhau tùy theo yêu cầu của vấn đề.

Trong phần Triển khai của cuốn sách này, chúng ta sẽ sử dụng CNN để phân loại và thử nhận dạng thực vật hoặc các loài của chúng.